



YISHUN INNOVA JUNIOR COLLEGE
JC 1 PROMOTIONAL EXAMINATION
Higher 2

CANDIDATE
NAME

CG

INDEX NO

COMPUTING

Paper 2 Practical

9569/02

3 Oct 2025

1 hour 50 minutes

Additional Materials:

Removable storage device with the following files:

- `PROMO_template.ipynb`
- `RECORDS.TXT`
- `TASK5-1.TXT`
- `TASK5-2.TXT`
- `bakery.db`
- `module.py`

READ THESE INSTRUCTIONS FIRST

Answer **all** questions.

All tasks must be done in the computer laboratory. You are not allowed to bring in or take out any pieces of work or materials on paper or electronic media or in any other form.

Approved calculators are allowed.

Save each task as it is completed.

The use of built-in functions, where appropriate, is allowed for this paper unless stated otherwise.

The number of marks is given in brackets [] at the end of each task.

The total number of marks for this paper is **60**.

Instruction to candidates:

Your program code and output for all the questions should be saved in a single file and downloaded as

PROMO_<your class>_<your name>.ipynb

For each of the sub-tasks, add a comment statement, at the beginning of the code using the hash symbol '#', to indicate the sub-task the program code belongs to, for example:

In [1]:

#Task 1.1
Program Code

Output:

In [2]:

#Task 1.2
Program Code

Output:

In [3]:

#Task 1.3
Program Code

Output:

- 1** In a diving championship, a panel of nine judges each submits a score between 0 and 10, both inclusively, after observing the diver's execution of the dive. The scores will be provided to the Scorer to enter into the computer system.

Task 1.1

Write program code for the function `submit()` to prompt the Scorer to input the nine scores, one by one, and return a list containing all the scores. [4]

To minimise potential bias and prevent any single extreme score from disproportionately affecting the result, the two highest and two lowest scores are discarded.

Task 1.2

Modify the program code for the function `submit()`, written in Task 1.1, to remove the two highest and two lowest scores before returning the list of the remaining five scores. [2]

- 2** A software developer is creating a utility function to read data from text files. The field names are listed in the header row separated by commas `,`.

Task 2.1

Write program code for a function `openfile(filename)` to:

- read the data in the text file `filename`
- skip the header row
- separate the data in each row in the file and store them as a record in a tuple
- return a list containing all the record tuples. [6]

Task 2.2

Write program code to call the function `openfile(filename)` to read the `RECORDS.TXT` file. [1]

- 3 A fish farm begins with an initial population of 100 fish. Every month, the fish farmer harvests a constant number of fishes, denoted by the parameter c . The fish population follows a recurrence relation that models both natural growth and harvesting:

$$P_n = 1.2 \times P_{n-1} - c$$

where P_n is the fish population in the current month and P_{n-1} is the population in the previous month.

Task 3

Write program code for the recursive function `population(c, m)` that takes in the parameter c for the constant number of fishes harvested per month, and m for the number of months elapsed.

The program code should:

- use the recurrence relation given to compute the fish population after some months have elapsed
- apply the `round()` function to the population to obtain a whole number at each recurrence step of the calculation
- return the fish population after m months have elapsed
- assume that when $m=0$, the initial population of fishes is 100.

Calculate the population if the fisherman harvests 15 fish each month over a period of 12 months. [4]

- 4 A computer program is used to manage the print queue for a printer on a local area network (LAN). The print queue behaves as a queue data structure.

When a user sends a print job from a terminal on the LAN, the program will send a tuple with the following information to the print queue:

- the user's terminal number
- filename
- the file size which is an integer in Kbytes.

An empty print queue `pqueue` has been initialised.

The following print jobs are to be sent to the print queue:

```
jobs = (('terminal1', 'Proposal1.doc', '583'),
        ('terminal5', 'Diagram.jpg', '351'),
        ('terminal2', 'Meeting.doc', '43'),
        ('terminal1', 'Proposal2.doc', '592'))
```

Task 4.1

Write program code for the function `send(tup)` to add the jobs in `tup` to the print queue `pqueue` and return an integer indicating the number of jobs in the print queue. [3]

Task 4.2

Write program code for the function `pprint(n)` to remove `n` number of jobs from the print queue `pqueue` and return an integer indicating the number of jobs in the print queue.

Display an output statement `'The print queue is empty.'` when the queue is empty. [5]

- 5 The Quicksort algorithm is a recursive sorting technique that arranges elements in a sequence by repeatedly partitioning the sequence around a selected pivot value. The incomplete pseudocode for the function `Partition()` is provided in the file `TASK5-1.TXT`, and the missing parts of the algorithm are labelled A, B and C.

Task 5.1

Complete the pseudocode by writing the missing lines of codes.

Write program code for the function `Partition()` based on the provided pseudocode.

[5]

The incomplete pseudocode for the function `Quicksort()` and a helper function `QuickSortHelper()` are provided in the file `TASK5-2.TXT`. This helper function performs a single partition step and recursively sorts the left and right sub-arrays. The missing parts of the algorithm are labelled A and B.

Task 5.2

Write program code for the function `QuickSort()` and the recursive function `QuickSortHelper()` based on the provided pseudocode.

[2]

- 6 The following list `lstnum` contains a sequence of numbers sorted in ascending order:
- ```
lstnum = [2, 5, 12, 20, 35, 70, 99, 105, 225, 364, 600,
 930, 1222, 2002, 2502, 3010, 4666]
```

#### Task 6

Write program code for the iterative function `binarysearch(lst, item)` that:

- performs a binary search on the sorted list `lst` to determine whether or not the value `item` exists in the list
- returns `True` if `item` is found. Otherwise, returns `False`.

Use the function `binarysearch()` to determine whether or not the number 12 exists in the list `lstnum`.

[4]

- 7 A furniture store intends to implement a hash table to manage its inventory. A custom hash function is used to determine the index at which each furniture item should be stored in the hash table based on the name of the furniture.

### Task 7.1

Write program code for a hash function `hashfn(string)` that

- iterates each character of the input string `string`
- multiplies the ASCII value of the character by  $17^{(i+1)}$  where  $i$  is the index position of the character (starting from 0)
- returns the total modulo 25 [2]

For example, the hash value for the furniture named 'Guv' is as follows:

$$\begin{aligned}\text{hashfn('Guv')} &= (71 \times 17^{(0+1)} + 117 \times 17^{(1+1)} + 118 \times 17^{(2+1)}) \% 25 \\ &= 4\end{aligned}$$

### Task 7.2

Write program code for the function `hashtable(lst)` that takes in a list `lst` containing the names of the furniture. It should:

- create a hash table of size 25
- use the `hashfn(string)` function written in Task 7.1 to compute the index and store each string in the correct position in the hash table
- resolve any collision using separate chaining
- returns the populated hash table.

```
names = ["Snuvla", "Tordenk", "Blommig", "Kappar", "Fastig",
 "Lovkant", "Dromtorp", "Kullstad", "Vanklig", "Mornbak",
 "Trolka", "Gursnad", "Norby", "Piltra", "Somnig",
 "Frastorp", "Dolmig", "Halska", "Skurna", "Valkom",
 "Krundal", "Ornsta", "Flaktig", "Jornka", "Grynskar",
 "Stubbrod", "Barnig", "Zalvar", "Tallhov", "Holmbrak"]
```

Populate the above list of names into the hash table and print the hash table.

[6]

- 8 The arrangement of keys on a QWERTY keyboard was designed to slow the typists down so as to prevent the mechanical keys from jamming.

A new keyboard layout can be designed based on the letter frequency in our common English usage.

A 200-word passage is provided in the `module.py` file. Run the following program code to print the passage:

```
from module import *
passage = sample()
print(passage)
```

#### Task 8.1

Write program code to count the number of times each of the 26 letters in the English alphabet from 'a' to 'z' appear in the passage and store the information in a dictionary.

[3]

#### Task 8.2

Write program code to output a tabulated frequency table similar to the following:

| Letter | Count | Frequency (%) |
|--------|-------|---------------|
| A      | 73    | 8.01          |
| b      | 7     | 0.77          |
| c      | 27    | 2.96          |
| ...    |       |               |
| ...    |       |               |
| x      | 6     | 0.66          |
| y      | 14    | 1.54          |
| z      | 7     | 0.77          |

[5]

- 9 A bakery uses a relational database to manage its daily operations. The system tracks a list of products on the menu, the inventory and the daily sales transactions.

One of the table products in the database bakery.db contains the following fields:

Name, Description, UnitPrice and Quantity

**Task 9.1**

Write SQL Query command to display all the names and descriptions of all the products sold in the bakery shop. [2]

**Task 9.2**

Write SQL Query command to display the names and unit prices of all products with a unit price of \$5.00 or higher. [2]

The bakery prepares the pastries according to the quantities stated in the products table.

**Task 9.3**

Write SQL command to display the total revenue if all the products are sold. [2]

**Task 9.4**

A new product is to be added to the menu with the following information:

- Name: Sourdough Loaf
- Description: A rustic, tangy bread with a chewy inside and a thick, crunchy crust.
- Unit Price: \$7.00
- Quantity: 30

Write SQL command to insert a new record into the products table in the bakery database. [2]

**END OF PAPER**

**BLANK PAGE**